High-Level Design (HLD)

Architecture Specification: RAG-Based Support Assistant with LangGraph & HITL

1. System Overview

Problem Definition: Enterprise customer support operations suffer from a high volume of repetitive queries, leading to bottlenecked response times. Existing automation solutions (standard LLMs) are prone to hallucinations regarding specific proprietary policies and lack the capability to safely hand off complex edge-cases to human operators without losing conversation context.

Scope of the System: The system is a Tier-1 hybrid AI resolution agent. It operates within a tightly controlled Retrieval-Augmented Generation (RAG) framework, using a cyclic state machine architecture to query internal knowledge bases. Crucially, the scope includes a Human-in-the-Loop (HITL) interrupt mechanism that pauses execution on low-confidence generations, routing the full thread state to a human operator for seamless resolution.

2. Architecture Diagram

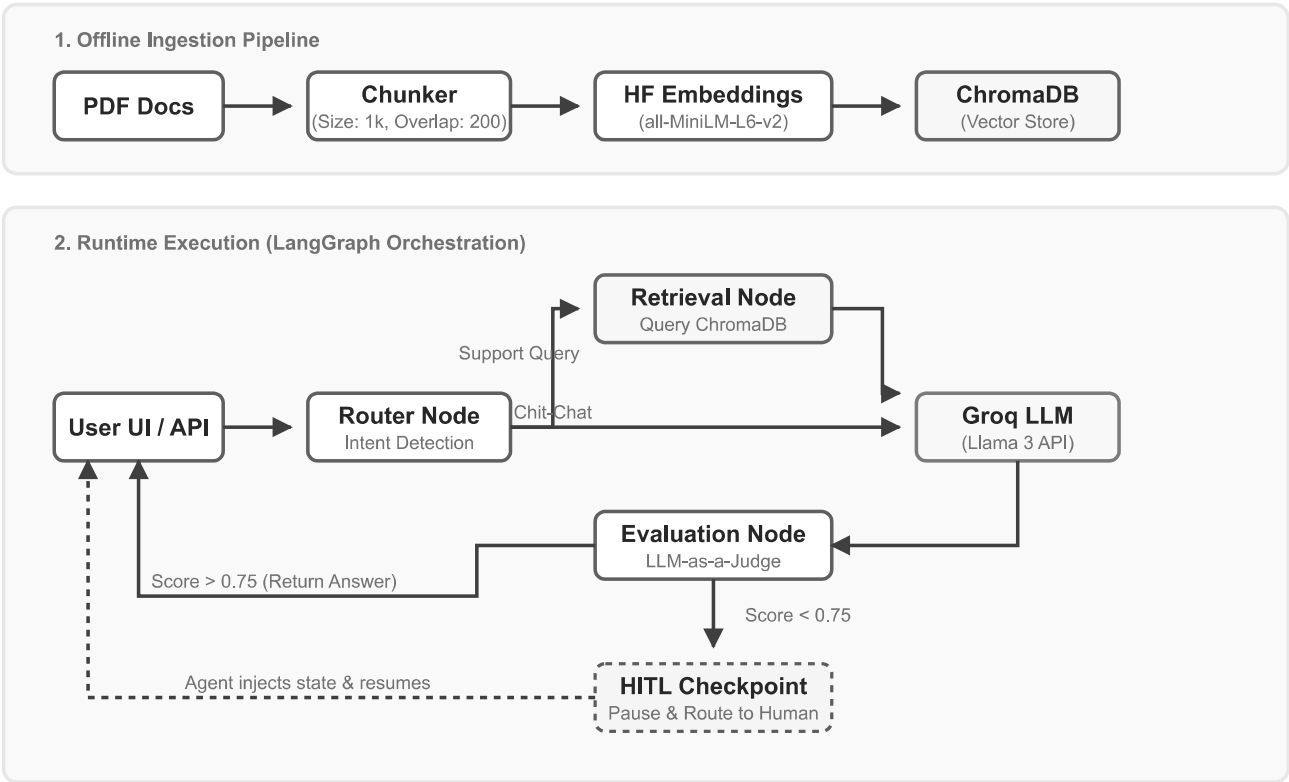

Figure 1.0: Component Architecture and State Routing Logic

3. Component Description & Deep Analysis

Component	Implementation	Deep Technical Analysis & Rationale
Document Loader	`PyPDFLoader`	Extracts raw binary PDF data into unstructured text objects. Retains critical metadata (page numbers, source path) which is injected into the vector store to provide citation capabilities in the final LLM prompt.
Chunking Strategy	`RecursiveCharacterTextSplitter` *Size: 1000, Overlap: 200*	**Deep Analysis:** Semantic integrity is paramount. Splitting strictly by character count cuts sentences in half, destroying context. The recursive splitter attempts to break on paragraphs (`\n\n`), then sentences (`.`), falling back to words. The 200-character overlap ensures that a concept split across two chunks remains mathematically linkable during vector similarity search.
Embedding Model	`HuggingFaceEmbeddings` *(all-MiniLM-L6-v2)*	Maps text into a 384-dimensional dense vector space. **Rationale:** Rather than relying on OpenAI's `text-embedding-3-small` (1536 dims), we utilize a local, open-source model. It is mathematically lightweight enough to run on standard CPUs, eliminating external API ingestion costs and significantly reducing network latency during the retrieval phase.
Vector Store	`ChromaDB`	Operates as a local SQLite-backed vector database using HNSW (Hierarchical Navigable Small World) indexing for fast nearest-neighbor lookups.
LLM Engine	`ChatGroq` *(llama-3.1-8b-instant)*	**Deep Analysis:** Groq utilizes specialized LPUs (Language Processing Units) rather than GPUs, achieving 800+ tokens/second. For a customer-facing support bot, perceived latency is the primary UX constraint. Llama 3.1 8B offers a perfect intersection of fast inference and high instruction-following capability for RAG prompts.
Workflow Orchestration	`LangGraph (StateGraph)`	Standard LangChain constructs (Runnables/Pipelines) are Directed Acyclic Graphs (DAGs) and cannot loop or pause. LangGraph introduces a cyclic state machine. A centralized `AgentState` TypedDict is mutated sequentially by nodes, allowing the system to pause execution natively.
HITL Module	`SqliteSaver` Checkpointer	Utilizes LangGraph's `interrupt_before` methodology. When triggered, the thread's memory is serialized to a persistent DB. The thread remains dormant until a human agent makes a POST request to `graph.update_state()` , injecting a `HumanMessage` payload.

4. Data Flow: PDF to Answer Lifecycle

Phase 1: Knowledge Ingestion (Offline)

1. **Load:** The system scans the `/data/raw` directory and loads `support_kb.pdf`.
2. **Transform:** Text is stripped of formatting anomalies and chunked into overlapping 1000-character arrays.
3. **Embed & Index:** Each chunk is passed through the HF model, resulting in a 384-dimension vector. The vector, original text, and metadata are committed to the Chroma DB local directory.

Phase 2: Query Execution (Online Runtime)

1. **Initialization:** User sends a query. A LangGraph thread is spawned with a unique `thread_id`.
2. **Intent Routing:** The LLM analyzes the query. If it identifies small-talk, it routes directly to the Generation Node. If it identifies a product issue, it routes to the Retrieval Node.
3. **Retrieval:** The user query is vectorized using the exact same `all-MiniLM-L6-v2` model. ChromaDB performs a Cosine Similarity search, returning the top 3 (`k=3`) closest vectors.
4. **Generation:** The LLM is prompted with strict constraints: "*Answer the question using ONLY the provided context.*"
5. **Evaluation & Guardrails:** The output is passed to an Evaluation Node (LLM-as-a-judge). It assigns a confidence score (0.0 to 1.0).
   - If Score ≥ 0.75: The answer is streamed back to the user UI.
   - If Score < 0.75: The graph invokes the HITL interrupt. The state is serialized to the checkpointer, and a webhook alerts the human support team.

5. Technology Choices Justification

- **Why ChromaDB over Pinecone/Qdrant?** For this project's current scope, an in-memory/local SQLite vector database provides zero network latency, zero cloud infrastructure cost, and absolute data privacy. Cloud-native vector DBs introduce unnecessary complexity for small to medium knowledge bases.
- **Why LangGraph over standard LangChain?** Enterprise bots cannot afford to be linear. If an LLM hallucinates, it needs to self-correct (loop). If it doesn't know the answer, it needs to wait for a human (pause). LangGraph provides state persistence and cyclic capabilities required for these advanced agentic behaviors.
- **Why HuggingFace Embeddings?** Transitioning from `OpenAIEmbeddings` to local HF models decouples the core infrastructure from closed-source ecosystems, prevents API rate-limiting during large batch ingestions, and keeps operating costs near zero.

6. Scalability Considerations

1. Handling Large Documents (Data Scale)
As the internal knowledge base grows beyond gigabytes, local ChromaDB instances will encounter RAM bottlenecks during similarity calculations. *Mitigation strategy:* Migrate the vector store implementation to a managed, distributed system like Pinecone or Weaviate, utilizing Approximate Nearest Neighbor (ANN) indexing at scale.

2. Increasing Query Load (Concurrency Scale)
The current architecture processes LangGraph threads in-memory. High concurrency will lead to race conditions or memory overflow. *Mitigation strategy:* Deploy the LangGraph application via a robust ASGI server (e.g., Uvicorn/FastAPI) and transition the HITL `SqliteSaver` checkpointer to a highly available Postgres database (using `PostgresSaver`) to manage distributed concurrent state.

3. Latency Concerns (Compute Scale)
Routing and Evaluation nodes double the total LLM API calls per query, introducing compounding latency. *Mitigation strategy:* Implement Semantic Caching (e.g., Redis). If a user asks "How do I reset my password", and the embedded query hits a 0.99 cosine similarity with a previously answered cached query, the system bypasses LangGraph entirely and returns the cached answer instantly.